Guía de estudio de PhishWarden MJ

BLOQUE 1 DE 9 | Lectura sugerida 3 minutos

Preparación para la exposición del 17 de octubre de 2026. Estado del código revisado el 16 de octubre de 2026.

PhishWarden MJ es una app móvil de aprendizaje para reconocer intentos de phishing. El avance incluye ingreso de empleados mediante NestJS, una notificación local real de Android, una microlección con pregunta y una pantalla de perfil en claro y oscuro. El perfil todavía no recibe estadísticas y la práctica no guarda resultados en el servidor.

La explicación que debes recordar

"Desarrollamos el cliente móvil con Flutter y Dart. Organizamos el código por funcionalidades y usamos MVVM sencillo junto con repositorios. Los widgets muestran la interfaz; los ViewModels controlan su estado; los repositorios obtienen los datos. Flutter se comunica con NestJS y el backend consulta PostgreSQL mediante Prisma."

Cómo repasar en el tiempo disponible

Tiempo	Qué estudiar
30 minutos	Lee los bloques 1 a 5 en 18 minutos; el 6 en 3 minutos; el guion del 8 en 5 minutos y las preguntas del 9 en 4 minutos. Practica después si dispones de más tiempo.
60 minutos	Dedica 35 minutos a los nueve bloques, 15 minutos a ejecutar el recorrido del bloque 7 y 10 minutos a explicar el guion sin leer.

Recorrido de esta guía

1. Resumen y plan de repaso. 2. Arquitectura. 3. Carpetas y archivos. 4. Ingreso real. 5. Notificación y microlección. 6. Perfil y temas. 7. Ejecución y pruebas. 8. Guion de exposición. 9. Preguntas y autoevaluación.

Tres límites que debes mencionar con claridad

El login actual valida correo y código, pero no entrega un token de sesión. La notificación la genera el celular; aún no llega desde un servidor. Los números de los mockups no son datos reales del empleado y no se copian al perfil.

Arquitectura y responsabilidades

BLOQUE 2 DE 9 | Lectura sugerida 5 minutos

La organización es por funcionalidades, también llamada feature first: auth, training y profile agrupan el código de cada función. Dentro de cada una separamos presentación y datos. MVVM significa Model View ViewModel, o modelo, vista y modelo de vista. Lo combinamos con el patrón Repository.

Pieza	Responsabilidad	Ejemplo real
View	Dibuja y recoge acciones del usuario.	WelcomeScreen, ProfileScreen
ViewModel	Mantiene carga, errores y estado. Solicita operaciones.	WelcomeViewModel, LessonViewModel
Modelo	Representa datos con tipos de Dart.	LinkResult, Lesson, EmployeeProfile
Repositorio	Define y resuelve el acceso a datos.	AuthRepository y ApiAuthRepository
Servicio compartido	Resuelve una capacidad técnica reutilizable.	ApiClient, NotificationService

Qué significa MVVM aquí

La vista escucha al ViewModel mediante ListenableBuilder. El ViewModel extiende ChangeNotifier y llama notifyListeners() cuando cambia el estado. No usamos Provider, Riverpod ni Bloc: el mecanismo de notificación viene con Flutter. Los modelos de datos son clases Dart; no hay obligación de tener una carpeta llamada model para cada una.

Qué aporta Repository

AuthRepository define la operación que necesita el ingreso. ApiAuthRepository la implementa usando HTTP y MockAuthRepository permite probar con respuestas controladas. El ViewModel recibe la interfaz y no decide si los datos vienen de una API o de una prueba. No existe una sustitución automática por datos ficticios si el servidor falla.

```
Vista → ViewModel → Repositorio → ApiClient
                                      ↓
                          NestJS → Prisma → PostgreSQL
```

Por qué app.dart conecta las piezas

app.dart crea los repositorios y servicios, los entrega por constructor y coordina las rutas. Esto es inyección de dependencias manual: un objeto recibe sus colaboradores en vez de crearlos todos por su cuenta. Cambiar un repositorio no exige introducir HTTP en los widgets.

Relación con la arquitectura del backend

El módulo auth del backend separa domain, application e infrastructure. LoginUseCase depende de AuthRepositoryPort y PrismaAuthRepository implementa ese puerto. Es una separación de puertos y adaptadores. Flutter no tiene que copiar esas carpetas: ambos lados se comunican por el contrato HTTP y JSON.

Mapa del proyecto y archivos

BLOQUE 3 DE 9 | Lectura sugerida 5 minutos

```
lib/
  main.dart
  app/app.dart
  core/
    theme/app_theme.dart
    network/api_config.dart y api_client.dart
    notifications/notification_service.dart
  features/
    auth/
      data/models/link_models.dart
      data/  interfaces y repositorios de ingreso
      presentation/  bienvenida y su ViewModel
    training/
      data/lesson_repository.dart
      presentation/  práctica, lección y ViewModels
    profile/
      data/profile_repository.dart
      presentation/  perfil y su ViewModel
```

Puntos de entrada y código compartido

main.dart prepara Flutter, inicializa las notificaciones y ejecuta runApp(). Si esa inicialización falla, permite abrir la bienvenida. app.dart crea MaterialApp, configura temas y navegación y usa ApiAuthRepository en la ejecución normal. core contiene lo compartido entre funcionalidades; no es una pantalla.

Dónde cambiar cada cosa

Necesidad	Archivo o carpeta
Cambiar el formulario de ingreso	features/auth/presentation/welcome_screen.dart
Cambiar reglas y estado del ingreso	features/auth/presentation/welcome_view_model.dart
Adaptar el JSON del login	features/auth/data/api_auth_repository.dart
Cambiar URL o tratamiento HTTP	core/network/
Editar consejos y pregunta locales	features/training/data/lesson_repository.dart
Cambiar diseño del perfil	features/profile/presentation/profile_screen.dart
Cambiar colores compartidos	core/theme/app_theme.dart

Carpetas y archivos fuera de lib

android/ contiene permisos, recursos y compilación nativa. test/ verifica lógica y widgets con datos controlados. integration_test/ contiene el recorrido Android contra NestJS. tool/check_login.dart permite consultar el login desde la terminal. pubspec.yaml declara paquetes; pubspec.lock fija las versiones resueltas. build/ y .dart_tool/ son generadas: no son el lugar para escribir funciones.

access_screen.dart conserva una vista provisional anterior. El ingreso actual abre ProfileScreen. Las carpetas ios/, web/ y windows/ no significan que hayamos validado las funciones móviles en esas plataformas.

Ingreso del empleado paso a paso

BLOQUE 4 DE 9 | Lectura sugerida 5 minutos

La cuenta del empleado debe existir previamente en la empresa. Tu compañero crea las cuentas. La app no registra empresas ni empleados y este flujo móvil no pide contraseña.

Recorrido al pulsar Ingresar

1. WelcomeScreen recoge los TextEditingController y valida el formulario. Se revisa que el código no esté vacío y que el correo tenga un formato básico válido; esto no demuestra que la cuenta exista.

2. WelcomeViewModel.submit() evita envíos simultáneos, activa isLoading y llama al repositorio. notifyListeners() permite que la vista muestre la carga o un mensaje de error.

3. ApiAuthRepository limpia espacios, normaliza el correo a minúsculas y transforma companyCode en codigoEmpresa. ApiClient envía JSON con un límite de espera de 15 segundos.

```
POST /auth/login
{
  "email": "empleado@empresa.com",
  "codigoEmpresa": "CODIGO_EMPRESA"
}
```

4. AuthController recibe la solicitud en NestJS. LoginUseCase busca la empresa por código y después un empleado con ese correo y el ID de la empresa. PrismaAuthRepository consulta la base de datos.

```
{
  "empresa": "Nombre de empresa",
  "user": { "id": "ID_EMPLEADO", "email": "empleado@empresa.com" }
}
```

5. La respuesta incluye también un mensaje. La app exige empresa, ID y correo válidos para crear LinkResult. NestJS devuelve 201 en el contrato revisado; el cliente admite los códigos 2xx. app.dart recibe el resultado y abre Mi perfil, retirando la bienvenida del historial de navegación.

Errores y significado de la sesión actual

Un rechazo 401 no abre el perfil. Un timeout, una desconexión, un error del servidor o un JSON incompleto se convierten en mensajes para reintentar. La app no consulta Supabase directamente y no contiene su contraseña de base de datos.

El método se llama linkDevice por el primer prototipo, pero ahora ejecuta el login: todavía no vincula un dispositivo ni registra un token push. El servidor no emite JWT ni comprueba la propiedad del correo mediante un código enviado a ese correo. Cerrar sesión elimina el acceso local; no revoca una sesión remota. La búsqueda de empleado revisada tampoco filtra su estado activo. Son límites pendientes del backend.

Notificaciones y microlección

BLOQUE 5 DE 9 | Lectura sugerida 4 minutos

Qué funciona de verdad en Android

NotificationService usa flutter_local_notifications para solicitar permiso y mostrar un aviso en la bandeja del sistema. El canal se llama microlearning y agrupa los avisos de microlecciones. NotificationPracticeViewModel controla el envío, la espera y el mensaje si el permiso está desactivado.

El aviso lleva un payload con el identificador phishing-email. El payload es un dato que viaja con la notificación; no es el texto completo de una lección ni un token de autenticación.

```
Botón de práctica → NotificationService → Android
Tocar aviso → pendingLesson → app.dart
           → LessonScreen → LessonRepository
```

Cómo abre la lección correcta

Si la app ya estaba abierta, onDidReceiveNotificationResponse recibe el toque. Si el aviso inicia una app terminada, getNotificationAppLaunchDetails() recupera ese dato. El servicio publica el ID en pendingLesson, un ValueNotifier. app.dart lo escucha y espera a que Navigator esté construido antes de abrir la ruta /lesson/phishing-email.

El identificador se coordinará con el backend cuando existan mensajes remotos. Un ID desconocido muestra que no se encontró la lección. El acceso directo actual es adecuado para este contenido local de práctica; al incorporar contenido privado habrá que comprobar la sesión antes de abrirlo.

Contenido y evaluación local

LocalLessonRepository contiene una sola lección: Reconoce un correo de phishing. Entrega consejos, pregunta, alternativas, índice correcto y explicación. Sus datos son locales y no se descargan de PostgreSQL.

LessonViewModel carga la lección, guarda selectedAnswer y evita responder otra vez después de enviar. isCorrect compara el índice seleccionado con correctAnswer. El índice 1 identifica la segunda alternativa porque las listas empiezan en cero. LessonScreen muestra el resultado y la explicación.

Notificación local y push remoto

La notificación local se genera dentro de la app y se muestra mediante Android. Un push remoto requerirá un servicio de mensajería, registrar el token del dispositivo en NestJS y recibir el ID de lección desde el servidor. Eso todavía no se implementó. Tener permiso de notificaciones no demuestra que un servidor ya pueda enviarlas.

El resultado de la pregunta existe únicamente en memoria durante esa pantalla. Al salir se reinicia: no se guardan respuestas, no se suman puntos y no se recalcula el riesgo.

Perfil temas y estado del avance

BLOQUE 6 DE 9 | Lectura sugerida 4 minutos

La pantalla Mi perfil sigue la estructura y los colores de las imágenes claras y oscuras. Los valores de Lucía Mora en el mockup son ejemplos visuales; el login real solo devuelve ID, correo y empresa.

Cómo llegan los datos al perfil

SessionProfileRepository recibe LinkResult y construye EmployeeProfile. ProfileViewModel controla carga, errores y reintento. El nombre, departamento, riesgo, puntos, racha y contadores son opcionales. Un int? admite un entero o null; aquí null significa dato no disponible, no cero ni riesgo bajo.

Por eso el perfil muestra el correo y la empresa reales, un saludo genérico y guiones o Sin datos en las métricas. Las categorías de medallas son referencias del diseño y no logros ganados. Tener columnas en PostgreSQL no hace que Flutter pueda leerlas: falta una ruta autorizada que entregue esos datos.

Temas y navegación

app_theme.dart define los temas compartidos. app.dart mantiene un ValueNotifier<ThemeMode>; ValueListenableBuilder reconstruye MaterialApp cuando cambia. Claro y Oscuro se eligen manualmente; Sistema sigue al dispositivo. La preferencia no se guarda en disco y al reiniciar vuelve a Sistema.

ProfileScreen recibe callbacks onLesson, onTraining y onLogout. Un callback es una función que la pantalla ejecuta para comunicar una acción; app.dart decide a qué ruta ir. Ajustes abre un panel pequeño de apariencia y salida, no toda la pantalla 12 del diseño.

Función	Estado actual
Ingreso con NestJS	Implementado con código y correo de empleado existente
Perfil y tema claro u oscuro	Implementados; estadísticas todavía sin datos
Aviso Android y apertura de lección	Implementación local con ID de lección
Pregunta de refuerzo	Evaluación local, sin guardar resultados
Capacitación en barra inferior	Abre la práctica de notificaciones existente
Centro completo de capacitación	Pendiente; corresponde al siguiente paso visual
Historial, logros y Ranking	Accesos deshabilitados y desarrollo pendiente
Push remoto y sesión con token	Pendientes de integración con backend

Diseño y requisitos académicos

Las imágenes orientan el aspecto. RF06, RF07, RF08 y RF09 se han usado como referencia de alcance a partir del resumen del trabajo. No presentes cada detalle del mockup como una exigencia textual de la evaluación. Para citar una condición exacta, consulta el PDF original.

Cómo ejecutar y demostrar la app

BLOQUE 7 DE 9 | Lectura sugerida 5 minutos

Prepara tres terminales: servidor, emulador y Flutter. Mantén disponibles Internet para la base remota y el backend configurado por tu compañero. No abras otra instancia del servidor si el puerto 3000 ya está ocupado.

Primero el backend

En la carpeta phishwarden-backend/phishwarden-backend-nj, con sus dependencias y .env ya preparados:

```
npm run build
node --env-file=.env dist/main.js
```

build compila TypeScript a JavaScript; node ejecuta ese resultado y carga .env. Deja la terminal abierta. No copies .env a Flutter. Un error Prisma P1001 es una conexión fallida a la base y debe resolverse en el servidor antes de probar el ingreso.

Después el emulador

En Android Studio abre Device Manager y pulsa iniciar en Medium_Phone_API_37.0. Si queda offline, cierra esa instancia y usa Cold Boot. También puedes abrirlo desde una terminal:

```
& "$env:LOCALAPPDATA\Android\Sdk\emulator\emulator.exe" `
  -avd Medium_Phone_API_37.0 -no-snapshot-load
```

Este arranque no carga la instantánea guardada. Espera a ver Android listo. No inicies dos veces el mismo dispositivo. El carácter ` al final de una línea continúa el comando en PowerShell.

Por último Flutter desde phishwarden_movil

```
$env:FLUTTER_WINDOWS = 'false'
flutter pub get
flutter devices
flutter run -d emulator-5554 --no-pub `
  --dart-define=API_BASE_URL=http://10.0.2.2:3000
```

La variable evita en esta terminal la preparación del destino Windows que causó el problema de enlaces de plugins. pub get resuelve dependencias; devices da el ID real que debes usar; run compila, instala y abre. --no-pub reutiliza las dependencias y --dart-define fija la URL al compilar. 10.0.2.2 apunta a tu PC desde el emulador; localhost apuntaría al propio Android. Para un celular físico consulta el reenvío USB en PASO_03_LOGIN_REAL.md.

Demostración de cinco minutos

Ingresa con la cuenta de prueba, verifica correo y empresa, cambia de tema, abre la microlección y responde. Regresa y abre Capacitación; solicita el aviso y tócalo desde la bandeja. Termina con Ajustes → Cerrar sesión. Prueba antes de exponer: SALA2026 y mariano.silva@techcorp.com fueron comprobados el 13 de octubre; su disponibilidad actual depende del backend.

```
flutter analyze --no-pub
flutter test --no-pub
```

analyze revisa tipos y reglas de código; test ejecuta pruebas controladas sin necesitar base de datos. La integración real Android es una prueba aparte en integration_test/ y requiere servidor y emulador. Las pruebas de widgets no prueban por sí solas la bandeja nativa ni la red real.

Guion para exponer en cinco minutos

BLOQUE 8 DE 9 | Lectura sugerida 3 minutos

Presentación del problema y del avance

"Mi proyecto se llama PhishWarden MJ. Busca ayudar al empleado a reconocer mensajes de phishing mediante entrenamientos breves. Yo desarrollo la app móvil con Flutter y Dart y mi compañero desarrolla la API con NestJS. Hasta ahora tenemos el ingreso real, el perfil visual, una notificación local de Android y una microlección con pregunta."

Explicación de la arquitectura

"Organizamos lib por funcionalidades: auth, training y profile. En cada una separamos la interfaz y el estado del acceso a datos. Usamos MVVM sencillo: la pantalla dibuja, el ViewModel controla carga y errores y el repositorio obtiene la información. Los ViewModels usan ChangeNotifier para que la pantalla se actualice cuando algo cambia."

"Repository nos permite sustituir el acceso de prueba por una API sin escribir solicitudes dentro de los widgets. En app.dart elegimos las implementaciones y conectamos la navegación. En core dejamos el tema, el cliente HTTP y el servicio de notificaciones."

Explicación del ingreso durante la demostración

"El empleado ya debe estar creado por la empresa. Aquí escribo el código y correo. La validación local comprueba el formato; la validación de pertenencia la hace NestJS. El repositorio envía POST /auth/login y el backend consulta PostgreSQL con Prisma. Si responde con datos válidos, abro Mi perfil. Si falla, muestro un error y permito reintentar."

Explicación del perfil y de la microlección

"El correo y la empresa que vemos vienen del servidor. Riesgo, puntos y racha todavía no tienen un contrato de API, por eso no muestro las cifras de ejemplo del mockup. Puedo cambiar entre claro, oscuro y sistema. Al abrir la práctica, Android muestra una notificación real; al tocarla, el ID phishing-email abre la microlección correspondiente."

"La pregunta compara mi respuesta con la alternativa correcta y explica por qué. La lección y esta evaluación son locales. El resultado todavía no se envía al servidor ni modifica el riesgo del perfil."

Pruebas y trabajo siguiente

"Tenemos pruebas de validación, HTTP, estado y navegación. También se comprobó en Android el ingreso real, la salida y el rechazo de un código inválido el 13 de octubre. Como trabajo siguiente falta el centro de capacitación, acordar los datos del perfil, guardar respuestas, recibir push remoto y completar una sesión protegida con token."

Qué mostrar en VS Code si te lo piden

Abre app.dart para explicar conexiones; welcome_view_model.dart para enseñar isLoading y notifyListeners(); api_auth_repository.dart para enseñar el JSON; y lesson_repository.dart para mostrar qué datos siguen siendo locales. No es necesario leer los archivos completos ni memorizar todos los widgets privados.

Preguntas para defender el proyecto

BLOQUE 9 DE 9 | Lectura sugerida 5 minutos

Por qué Flutter y qué diferencia hay con Dart

Flutter es el framework que construye la interfaz y permite integrar capacidades del dispositivo. Dart es el lenguaje en que escribimos esa interfaz y su lógica. El trabajo actual se centra en Android; no afirmamos que todas las plataformas estén verificadas.

Qué es Future y por qué usamos async y await

Future representa un resultado que estará disponible después. async permite escribir una operación asíncrona y await espera su resultado sin bloquear la interacción como lo haría una tarea síncrona larga. Durante la espera mantenemos isLoading para informar al usuario.

Qué significan final dispose y mounted

final impide reasignar una variable, aunque el objeto pueda tener estado mutable. dispose libera controladores y listeners cuando ya no se necesitan. mounted permite comprobar que un State sigue en el árbol antes de usar su contexto tras una espera. El ViewModel también evita notificar después de liberarse.

Se está usando autenticación de Supabase

No. En el flujo actual Flutter llama a NestJS y Prisma consulta PostgreSQL alojado en Supabase. No se usa Supabase Auth en Flutter. El ingreso actual comprueba registros por código y correo; falta una sesión con token y protección de rutas para considerarlo completo.

Qué diferencia hay entre una interfaz y una implementación

La interfaz define lo que debe poder hacerse, por ejemplo buscar una lección por ID. La implementación define cómo hacerlo: LocalLessonRepository usa contenido local. Una futura implementación HTTP podrá respetar la misma interfaz.

Qué demuestra el perfil si no tiene estadísticas

Demuestra el flujo posterior al ingreso, la representación de identidad confirmada, los temas y los accesos funcionales. Los campos opcionales permiten integrar estadísticas después sin presentar datos desconocidos como reales.

Qué ocurre si la profesora pide historial o Ranking

Esas pantallas siguen pendientes y sus accesos están deshabilitados. El desarrollo se está haciendo por funciones pequeñas que se puedan ejecutar y explicar. El siguiente paso previsto es el centro de capacitación.

Última comprobación antes de salir

Sin leer, explica el recorrido desde Ingresar hasta PostgreSQL; señala un ViewModel y un repositorio; diferencia aviso local y push; explica por qué aparece Sin datos; y demuestra la salida. Si puedes hacerlo en cinco minutos, dominas el avance que vas a presentar.

Dónde ampliar el repaso

En la raíz del proyecto están PASO_02_ANDROID.md, PASO_03_LOGIN_REAL.md y PASO_04_PERFIL.md. La guía del paso 3 conserva información histórica: para el recorrido vigente, después del login se abre Mi perfil y la salida está en Ajustes. Esta guía reúne ese estado actual.